0355. Design Twitter

um | Category: Heap / Priority Queue | LeetCode: https://leetcode.com/problems/

Problem

- Support postTweet(user, tweetId), getNewsFeed(user) (10 most recent tweets from the user and followees), follow and unfollow.

Intuition

- Each user's tweets form a list in time order (newest last).
- The feed is a k-way merge of those lists by recency: push the newest tweet of each relevant user into a max-heap by timestamp, pop 10 times, pushing each popped tweet's predecessor.

Brute Force

- Gather all relevant tweets, sort by time, take 10: O(T log T).

Optimal Approach

- Global counter as a timestamp.
- Tweets: user → list of (time, id). follows: user → set.
- Feed: heap of (time, id, user, index); seed with each followee's (and own) latest; pop up to 10.

Worked Example

```
post(1,5)                  u1: [5]
feed(1) -> [5]
follow(1,2); post(2,6) u2: [6]
feed(1) -> [6,5]           heap merges u1 and u2 by time
unfollow(1,2)
feed(1) -> [5]
```

Java Solution

```java
public class Twitter {
    private int clock = 0;
    private final Map<Integer, List<int[]>> tweets = new HashMap<>(); // {time, id}
    private final Map<Integer, Set<Integer>> follows = new HashMap<>();

    public void postTweet(int userId, int tweetId) {
        tweets.computeIfAbsent(userId, k -> new ArrayList<>()).add(new int[]{clock++, tweetId});
    }

    public List<Integer> getNewsFeed(int userId) {
        // heap entries: {time, tweetId, user, index}
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[0] - a[0]);
        Set<Integer> users = new HashSet<>(follows.getOrDefault(userId, Set.of()));
        users.add(userId);
        for (int u : users) {
            List<int[]> l = tweets.get(u);
            if (l != null && !l.isEmpty()) {
                int i = l.size() - 1;
                pq.add(new int[]{l.get(i)[0], l.get(i)[1], u, i});
            }
        }
        List<Integer> feed = new ArrayList<>();
        while (!pq.isEmpty() && feed.size() < 10) {
            int[] e = pq.poll();
            feed.add(e[1]);
            if (e[3] > 0) {
                int[] prev = tweets.get(e[2]).get(e[3] - 1);
                pq.add(new int[]{prev[0], prev[1], e[2], e[3] - 1});
            }
        }
        return feed;
    }

    public void follow(int followerId, int followeeId) {
        if (followerId != followeeId) follows.computeIfAbsent(followerId, k -> new HashSet<>()).add(followeeId);
    }
```

Java Solution (2)

```
    public void unfollow(int followerId, int followeeId) {
        Set<Integer> s = follows.get(followerId);
        if (s != null) s.remove(followeeId);
    }
}
```

Complexity

- Post/follow/unfollow O(1); feed O(F + 10 log F) for F followees.

Pitfalls

- A user must not be able to unfollow themselves (their own tweets always appear).